

Agent Monitor — Documentação Técnica

Versão 3.0 | Maio 2026

Campo	Valor
Projeto	Monitoramento de Sistemas P2
Módulo	Agent — Discovery + Coleta + Identidade + Executável
Linguagem	Python 3.12
Ambiente Testado	Ubuntu Linux 24 (físico e virtualizado)

Sumário

- 1. Visão Geral
- 2. Estrutura de Diretórios
- 3. Fluxo de Execução
- 4. Módulo: global_information
- 5. Módulo: discovery
 - 5.1 CPU Discovery
 - 5.2 Memory Discovery
 - 5.3 Disk Discovery
 - 5.4 Network Discovery
 - 5.5 Motherboard Discovery
 - 5.6 System Discovery
 - 5.7 Tools Discovery
- 6. Módulo: coleta
- 7. Estrutura dos Payloads
- 8. Módulo: utils
- 9. Módulo: sender
- 10. Organização como Módulo Python

11. Geração do Executável com PyInstaller

12. Dependências

13. Resumo dos Módulos

1. Visão Geral

O Agent Monitor é um agente de monitoramento Linux escrito em Python 3.12. Ele executa duas funções principais:

Discovery — coleta uma vez, na inicialização, um inventário completo de hardware e sistema operacional do host.

Coleta Contínua (métricas) — a cada 5 segundos, envia métricas de uso de CPU, memória, disco, rede e logs do sistema.

O agente detecta automaticamente se está rodando em hardware físico ou em ambiente virtualizado (KVM, VMware, Xen, Hyper-V) e adapta a coleta conforme o que está disponível. Campos inexistentes em VMs (slots de RAM, S.M.A.R.T., speed de rede) são retornados como `null`, e uma seção `notes` no payload avisa o backend do motivo.

O agente é distribuído como um executável único gerado com PyInstaller (`--onefile`), sem dependência de Python instalado na máquina alvo.

2. Estrutura de Diretórios

```
1 Agent/
2   agent/
3     __init__.py
4     __main__.py           ← ponto de entrada principal
5     global_information/
6       global_information.py   ← identidade do agente e do host
7       cria_executavel.txt     ← comando de build do PyInstaller
8     discovery/
9       __init__.py
10      cpu_discovery/
11        cpu.py               ← roteador físico/virtual
12        cpu_physical.py
13        cpu_virtual.py
14      mem_discovery/
15        mem.py
16        mem_physical.py
17        mem_virtual.py
18      disk_discovery/
19        disk.py
20        disk_physical.py
21        disk_virtual.py
22      network_discovery/
23        network.py
24        network_physical.py
25        network_virtual.py
26      motherboard_discovery/
```

```

27     motherboard.py
28     motherboard_physical.py
29     system_discovery/
30     system.py
31     system_physical.py
32     system_virtual.py
33     tools_discovery/
34     tools.py                ← ponto de entrada do módulo
35     tools_checker.py        ← lógica de verificação e instala
36     tools_physical.py
37     tools_virtual.py
38     coleta/
39     collector.py            ← orquestrador de coleta contínuo
40     cpu_coleta/cpu.py
41     mem_coleta/mem.py
42     disk_coleta/disk.py
43     network_coleta/network.py
44     logs_coleta/logs.py
45     utils/
46     parsers.py              ← todos os parsers centralizado
47     sender.py               ← envio HTTP ao backend
48     serializer.py           ← serialização JSON
49     shell.py                ← execução de comandos shell
50

```

3. Fluxo de Execução

Ao ser iniciado, o agente segue este fluxo:

Etapa	Descrição
1	<code>build_global_information("discovery")</code> — resolve identidade (agent_id, host_id, hostname, primary_ip), detecta ambiente e persiste IDs em disco
2	<code>get_tools_info()</code> — executado primeiro dentro do discovery; verifica e instala dependências se necessário
3	<code>run_discovery()</code> — executa todos os módulos de discovery e monta o payload
4	Envia o payload de discovery ao backend via <code>send_data()</code> (atualmente impresso via <code>print</code>)

5

Loop infinito: a cada 5 segundos, `collect_all()` coleta métricas e envia payload de métricas

Flag `--install-deps` : passada na linha de comando, força a instalação de ferramentas ausentes sem prompt interativo. Detectada em `main()` via `sys.argv` e repassada para `get_tools_info(force_install=True)`.

Resiliência por isolamento: cada módulo de discovery é chamado via `_safe_collect()`, que captura exceções individualmente. Se um módulo falhar (ex: `dmidecode` sem permissão root), o restante do payload não é afetado — o campo fica com `{"error": "<mensagem>"}` e o agente continua normalmente.

```
1 def _safe_collect(name: str, fn, *args, **kwargs):
2     try:
3         return fn(*args, **kwargs)
4     except Exception as e:
5         print(f" Erro no discovery de '{name}': {e}")
6         return {"error": str(e)}
7
```

4. Módulo: `global_information`

Arquivo: `agent/global_information/global_information.py`

O que faz

Monta o bloco `global` que encabeça todos os payloads (discovery e métricas). É chamado uma vez por tipo de coleta. Centraliza a detecção de ambiente, evitando que cada módulo execute `lscpu` individualmente.

Funções internas

`_load_or_create_id(filename)` Lê o ID do arquivo em `~/.agent/<filename>`. Se não existir ou for inválido, gera um número de 5 dígitos (10000–99999) e salva. O ID sobrevive a reinicializações; só muda se o arquivo for deletado manualmente.

`_get_hostname()` Retorna o FQDN via `socket.getfqdn()`. Fallback: `socket.gethostname()`.

`_get_primary_ip()` Determina o IP primário conectando um socket UDP ao `8.8.8.8:80`. Nenhum pacote é enviado — a técnica consulta a tabela de roteamento do kernel para descobrir qual IP seria usado para alcançar a internet.

`_detect_environment()` Executa `lscpu` e verifica o campo `Hypervisor` `Vendor`. Se presente, o host é virtualizado. Este é o único lugar no agente onde `lscpu` é executado para detecção de ambiente.

`_build_notes(is_virtualized)` Gera lista de notas informativas para o backend sobre campos indisponíveis em VM. Em hardware físico, retorna lista vazia.

Payload gerado

```
1 {
2   "collection_type": "discovery",
3   "schema_version": "1.0",
4   "agent_id": "38472",
5   "host_id": "71203",
6   "hostname": "servidor-01.exemplo.com",
7   "primary_ip": "192.168.1.42",
8   "environment": {
9     "is_virtualized": true,
10    "hypervisor": "VMware"
11  },
12  "notes": [
13    "cpu topology reflects VM vCPU allocation, not physical cores",
14    "memory slots unavailable in virtualized environments",
15    "disk is virtual – smartctl data unavailable",
16    "network hardware fields (speed, driver, bus_info) unavailable in V",
17    "motherboard section absent in virtualized environments"
18  ]
19 }
20
```

Campo	Descrição
<code>collection_type</code>	"discovery" ou "metrics"
<code>schema_version</code>	Versão do schema — incrementar em breaking changes
<code>agent_id</code>	ID único do agente persistido em <code>~/.agent/agent_id.txt</code>
<code>host_id</code>	ID único do host persistido em <code>~/.agent/host_id.txt</code>
<code>hostname</code>	FQDN ou hostname do sistema
<code>primary_ip</code>	IP da rota de saída (interface preferida)
<code>environment</code>	<code>is_virtualized</code> + <code>hypervisor</code>

5. Módulo: discovery

O módulo de discovery é executado uma única vez após a inicialização e coleta o inventário completo de hardware e SO. Todos os sub-módulos seguem o mesmo padrão de detecção de ambiente: recebem `is_virtualized` como parâmetro (resolvido uma única vez em `global_information`) e delegam para o handler `_physical` ou `_virtual` correspondente.

5.1 CPU Discovery

Arquivos: `discovery/cpu_discovery/cpu.py`, `cpu_physical.py`, `cpu_virtual.py`

Fontes de dados:

- `lscpu` — modelo, fabricante, arquitetura, número de CPUs lógicas, threads por core, hipervisor
- `/proc/cpuinfo` — dados complementares por núcleo lógico
- `sysfs` (`/sys/devices/system/cpu/cpu0/cpufreq/`) — frequências base, máxima e mínima em kHz (convertidas para MHz)

Hardware físico: coleta `model_name`, `vendor`, `architecture`, `threads_per_core`, `cores_logical`, frequências (base, max, min). A frequência base é buscada primeiro no `sysfs` (mais preciso) e depois em `lscpu` como fallback.

Ambiente virtualizado: a topologia reflete a alocação de vCPUs do hipervisor, não os núcleos físicos reais. `threads_per_core` retorna `null` quando o valor é genérico (`"1"`), pois não é confiável em VMs. Cada valor de frequência inclui um campo `source` indicando de onde foi obtido (ex: `sysfs.base_frequency`, `proc_cpuinfo.cpu_mhz`).

Campo	Descrição
<code>model_name</code>	Modelo do processador
<code>vendor</code>	Fabricante (Intel, AMD...)
<code>architecture</code>	Arquitetura (x86_64, arm64...)
<code>topology.vcpus</code>	vCPUs alocados (somente VMs)

<code>topology.cores_logical</code>	CPUs lógicas (somente físico)
<code>frequency.base_mhz</code>	Frequência base com campo <code>source</code>
<code>frequency.max_mhz</code>	Frequência máxima com campo <code>source</code>
<code>frequency.min_mhz</code>	Frequência mínima com campo <code>source</code>

5.2 Memory Discovery

Arquivos: `discovery/mem_discovery/mem.py`, `mem_physical.py`, `mem_virtual.py`

Fontes de dados:

- `/proc/meminfo` — total, disponível, livre, buffers, cached, swap
- `dmidecode -t memory` (somente físico, requer root) — slots físicos de RAM, fabricante, tipo DDR, velocidade, serial

Hardware físico: retorna totais em bytes e uma lista detalhada de slots físicos. Cada slot inclui: localizador, banco, se está populado, tamanho, tipo DDR, velocidade em MHz, fabricante, part number, serial e fator de forma.

Ambiente virtualizado: `slots` retorna `null`. Além dos totais, inclui `buffers`, `cached` e informações de swap.

Campo	Descrição
<code>total_bytes</code>	RAM total em bytes
<code>swap_total_bytes</code>	Swap total em bytes
<code>slots</code>	<code>null</code> em VM; lista de módulos em físico

5.3 Disk Discovery

Arquivos: `discovery/disk_discovery/disk.py`, `disk_physical.py`, `disk_virtual.py`

Fontes de dados:

- `lsblk -J -b -o NAME,SIZE,TYPE,ROTA,MOUNTPOINT,FSTYPE,LABEL,UUID,MODEL,VENDOR,SERIAL,T`
`RAN,RM` — dispositivos de bloco em JSON nativo
- `smartctl -i -j` (somente físico, requer root) — modelo, serial, firmware, status S.M.A.R.T., horas de uso, temperatura

Hardware físico: cruza `lsblk` com `smartctl` para cada dispositivo. Loop devices (`type=loop`) são ignorados. Partições são coletadas recursivamente até `depth 4` para cobrir LVM sobre LUKS sobre partição. O tipo do disco (HDD/SSD) é determinado pelo `rotation_rate` do `smartctl` ou pelo flag `rota` do `lsblk`.

Ambiente virtualizado: usa apenas `lsblk`. Campos S.M.A.R.T. (`smart_passed`, `power_on_hours`, `temperature_celsius`) retornam `null`.

Resolução de tipo de disco (`resolve_disk_type`):

Prioridade	Condição	Resultado
1	<code>is_virtual=True</code>	"Virtual"
2	<code>rotation_rate=0</code>	"SSD"
3	<code>rotation_rate>0</code>	"HDD"
4	<code>rota_flag</code> do <code>lsblk</code>	"HDD" ou "SSD"
5	sem dados	"Unknown"

Resolução de interface (`resolve_disk_interface`): prioridade — campo `TRAN` do `lsblk` → `device.type` do `smartctl` → inferência pelo nome do modelo.

Campo	Descrição
<code>device</code>	Caminho do dispositivo (ex: <code>/dev/sda</code>)
<code>model</code> / <code>vendor</code> / <code>serial</code>	Identificação do disco
<code>type</code>	HDD SSD Virtual Unknown
<code>interface</code>	SATA NVMe SCSI USB ...
<code>form_factor</code>	2.5" M.2 ... (null em VM)

<code>size_bytes</code>	Tamanho do disco em bytes
<code>health</code>	<code>smart_passed</code> , <code>power_on_hours</code> , <code>temperature_celsius</code> (<code>null</code> em VM)
<code>partitions</code>	Lista recursiva de partições com <code>role</code> inferido

5.4 Network Discovery

Arquivos: `discovery/network_discovery/network.py` , `network_physical.py` , `network_virtual.py`

Fontes de dados:

- `ip -j addr show` — endereços IPv4/IPv6, prefixos, escopo
- `ip -j link show` — flags, MAC, MTU
- `ip -j route show default` — gateway padrão
- `sysfs ( /sys/class/net/<iface>/ )` — speed, duplex, driver, bus_info (somente físico)

Hardware físico: após construir as interfaces com `ip addr/link` , enriquece com `speed_mbps` , `duplex` , `driver` e `bus_info` via leitura direta do `sysfs`. `bus_info` é extraído do arquivo `uevent` do dispositivo (campo `PCI_SLOT_NAME`).

Ambiente virtualizado: usa apenas `ip addr` e `ip link` . Campos de hardware ficam ausentes do dict (não retornam `null` — são omitidos via `_normalize_network` no `__main__.py`).

Campo	Descrição
<code>total_interfaces</code>	Número de interfaces
<code>default_gateway</code>	<code>gateway</code> e <code>interface</code> da rota padrão
<code>interfaces[].name</code>	Nome da interface
<code>interfaces[].mac</code>	Endereço MAC
<code>interfaces[].mtu</code>	MTU

<code>interfaces[].state</code>	Estado operacional
<code>interfaces[].flags</code>	Flags da interface
<code>interfaces[].ipv4</code> / <code>ipv6</code>	Endereços com prefixo, escopo e broadcast
<code>interfaces[].speed_mbps</code>	Velocidade em Mbps (somente físico)
<code>interfaces[].duplex</code>	<code>full</code> <code>half</code> (somente físico)
<code>interfaces[].driver</code>	Driver do kernel (somente físico)
<code>interfaces[].bus_info</code>	Localização PCIe (somente físico)

5.5 Motherboard Discovery

Arquivos: `discovery/motherboard_discovery/motherboard.py`,
`motherboard_physical.py`

Fontes de dados:

- `dmidecode -t 0` — BIOS (fabricante, versão, data)
- `dmidecode -t 2` — baseboard (fabricante, modelo, serial, asset tag)
- `dmidecode -t 4` — sockets de CPU (quantidade e ocupação)
- `dmidecode -t 9` — slots de expansão PCIe
- `dmidecode -t 17` — contagem resumida de slots de RAM
- `lspci` — identificação do chipset via ISA/LPC bridge

Hardware físico: monta perfil completo da placa-mãe incluindo chipset, BIOS, sockets de CPU com status de ocupação, resumo de slots de RAM e lista de slots PCIe.

Ambiente virtualizado: o campo `motherboard` retorna `null` diretamente — o hipervisor não expõe informações úteis de placa-mãe.

Campo	Descrição
<code>manufacturer</code> / <code>product_name</code>	Fabricante e modelo da placa-mãe
<code>serial_number</code> / <code>asset_tag</code>	Identificação física
<code>chipset</code>	Chipset via <code>lspci</code> (<code>null</code> em VM)

<code>bios</code>	<code>vendor</code> , <code>version</code> , <code>release_date</code> , <code>revision</code>
<code>cpu_sockets</code>	Total de sockets e quantos populados (<code>null</code> em VM)
<code>ram_slots</code>	Total, usados e livres (<code>null</code> em VM)
<code>expansion_slots</code>	Lista de slots PCIe (ausente em VM)

5.6 System Discovery

Arquivos: `discovery/system_discovery/system.py` , `system_physical.py` ,
`system_virtual.py`

Fontes de dados:

- `/etc/os-release` — distribuição, versão, codinome
- `uname -a` — versão e release do kernel, arquitetura
- `hostname -f` — hostname completo
- `/proc/uptime` — tempo de atividade em segundos
- `timedatectl show --property=Timezone --value` — fuso horário

As informações de OS são idênticas para físico e virtualizado. O que difere é apenas o campo `is_virtualized` e `hypervisor` no bloco `global`.

Campo	Descrição
<code>hostname</code>	Nome completo do host
<code>os.name</code> / <code>pretty_name</code>	Nome e versão amigável do SO
<code>os.id</code> / <code>id_like</code>	Identificador interno e família
<code>os.version</code> / <code>version_id</code>	Versão e número da versão
<code>os.codename</code>	Codinome da distribuição (ex: <code>noble</code>)
<code>kernel.release</code> / <code>version</code> / <code>machine</code>	Dados do kernel

<code>timezone</code>	Fuso horário configurado
<code>uptime_seconds</code>	Tempo de atividade em segundos

5.7 Tools Discovery

Arquivos: `discovery/tools_discovery/tools.py`, `tools_checker.py`, `tools_physical.py`, `tools_virtual.py`

O que faz: verifica quais ferramentas externas estão disponíveis no host e, opcionalmente, instala as ausentes. É executado primeiro dentro do `run_discovery()`, antes dos demais módulos, para garantir que as dependências estejam presentes.

Ferramentas verificadas em físico: `lscpu`, `lsblk`, `ip`, `dmidecode`, `smartctl`, `ethtool`, `lspci`, `journalctl`, `timedatectl`

Ferramentas verificadas em VM: `lscpu`, `lsblk`, `ip`, `journalctl`, `timedatectl`
(ferramentas que requerem hardware físico são ignoradas)

Lógica de instalação (`tools_checker.py`):

1. Verifica quais ferramentas estão instaladas via `shutil.which`
2. Se houver ausentes: detecta o gerenciador de pacotes (`apt`, `dnf`, `pacman`, `zypper`)
3. Se for terminal interativo: pergunta ao usuário se deseja instalar
4. Se não for terminal interativo (serviço, pipe, cron): registra e segue sem instalar
5. Flag `--install-deps` força a instalação sem prompt em qualquer contexto
6. Re-verifica e atualiza o status após instalação

Campos que requerem root: `dmidecode`, `smartctl`. O campo `has_root` indica se o agente está rodando como root (`os.geteuid() == 0`).

Pacotes por gerenciador:

Ferramenta	apt	dnf	pacman	zypper
<code>dmidecode</code>	<code>dmidecode</code>	<code>dmidecode</code>	<code>dmidecode</code>	<code>dmidecode</code>
<code>smartctl</code>	<code>smartmontools</code>	<code>smartmontools</code>	<code>smartmontools</code>	<code>smartmontools</code>
<code>ethtool</code>	<code>ethtool</code>	<code>ethtool</code>	<code>ethtool</code>	<code>ethtool</code>
<code>lspci</code>	<code>pciutils</code>	<code>pciutils</code>	<code>pciutils</code>	<code>pciutils</code>

lsblk / lscpu	util- linux	util- linux	util- linux	util- linux
ip	iproute2	iproute	iproute2	iproute2
journalctl 1 / timedatectl	systemd	systemd	systemd	systemd

Payload gerado por ferramenta:

```

1  "lsblk": {
2      "installed": true,
3      "path":     "/usr/bin/lsblk",
4      "version":  "lsblk from util-linux 2.39.3",
5      "has_root": false,
6      "needs_root": false
7  }
8

```

6. Módulo: coleta

O módulo de coleta executa em loop infinito a cada 5 segundos. Todos os sub-módulos usam `psutil` para acessar informações do sistema de forma portátil.

Arquivo principal: coleta/collector.py

```

1  def collect_all():
2      return {
3          "cpu":      get_cpu_usage(),
4          "memory":   get_memory_usage(),
5          "disk":     get_disk_usage(),
6          "network":  get_network_usage(),
7          "logs":     get_logs()
8      }
9

```

6.1 CPU Coleta

Arquivo: coleta/cpu_coleta/cpu.py

Usa `psutil.cpu_percent(interval=1)`, que mede o uso durante 1 segundo. O intervalo garante medição precisa em vez de snapshot instantâneo.

Campo	Descrição
percent	Percentual de uso da CPU (0–100)

6.2 Memory Coleta

Arquivo: coleta/mem_coleta/mem.py

Usa `psutil.virtual_memory()`.

Campo	Descrição
<code>total</code>	RAM total em bytes
<code>used</code>	Memória utilizada em bytes
<code>percent</code>	Percentual de uso (0–100)

6.3 Disk Coleta

Arquivo: coleta/disk_coleta/disk.py

Usa `psutil.disk_usage('/')` para o ponto de montagem raiz.

Campo	Descrição
<code>total</code>	Espaço total em bytes
<code>used</code>	Espaço utilizado em bytes
<code>percent</code>	Percentual de uso (0–100)

6.4 Network Coleta

Arquivo: coleta/network_coleta/network.py

Usa `psutil.net_io_counters()`. Retorna contadores acumulados desde a inicialização do sistema — não são deltas. O backend é responsável por calcular a taxa entre amostras.

Campo	Descrição
<code>bytes_sent</code>	Total de bytes enviados (acumulado)
<code>bytes_recv</code>	Total de bytes recebidos (acumulado)

6.5 Logs Coleta

Arquivo: coleta/logs_coleta/logs.py

Lê as últimas 10 linhas de `/var/log/auth.log` . Registra eventos de autenticação: tentativas de login, falhas (possível brute force), escalonamento via `sudo` .

Em caso de falha de acesso, retorna lista com mensagem de erro descritiva.

Retorno	Descrição
<code>list[str]</code>	Últimas 10 linhas do <code>auth.log</code>
<code>list[str]</code>	Lista com mensagem de erro se inacessível

TODOs documentados no código: suporte a `syslog` , `dmesg` e `journalctl` ; controle de posição por inode para evitar duplicatas; detecção de rotação de logs; limitação por janela de tempo (ex: última semana ou 1000 eventos).

7. Estrutura dos Payloads

7.1 Payload de Discovery

Enviado uma vez na inicialização.

```
1 {
2   "global": {
3     "collection_type": "discovery",
4     "schema_version": "1.0",
5     "agent_id": "38472",
6     "host_id": "71203",
7     "hostname": "servidor-01.exemplo.com",
8     "primary_ip": "192.168.1.42",
9     "environment": {
10      "is_virtualized": false,
11      "hypervisor": null
12    },
13     "notes": []
14   },
15   "system": { "..."},
16   "cpu": { "..."},
17   "memory": { "..."},
18   "disk": { "..."},
19   "network": { "..."},
20   "motherboard": { "..."},
21   "tools": { "..."}
22 }
23
```

7.2 Payload de Métricas

Enviado a cada 5 segundos.

```
1 {
2   "global": {
3     "collection_type": "metrics",
4     "schema_version": "1.0",
5     "agent_id": "38472",
6     "host_id": "71203",
7     "hostname": "servidor-01.exemplo.com",
8     "primary_ip": "192.168.1.42",
9     "environment": { "... " },
10    "notes": []
11  },
12  "timestamp": 1746546782.12,
13  "data": {
14    "cpu": { "percent": 23.5 },
15    "memory": { "total": 8589934592, "used": 4294967296, "percent": 50
16    "disk": { "total": 107374182400, "used": 53687091200, "percent":
17    "network": { "bytes_sent": 1048576, "bytes_recv": 5242880 },
18    "logs": ["May 6 00:01:02 host sshd[1234]: Accepted ..."]
19  }
20 }
21
```

8. Módulo: utils

Arquivo: agent/utils/parsers.py

Todos os parsers e helpers estão centralizados em um único arquivo. Nenhuma chamada shell acontece aqui — apenas transformação de dados.

Seção	Funções	Usada por
Parsers de CPU	<code>parse_lscpu()</code> , <code>parse_cpufreqinfo()</code>	<code>cpu_*</code> , <code>global_informatio</code> <code>n</code>
Parsers de Memória	<code>parse_meminfo()</code> , <code>parse_meminfo_kb()</code> , <code>parse_dmidecode_m</code> <code>emory()</code> , <code>parse_dmi_size_to</code> <code>_mb()</code>	<code>mem_*</code>
Parsers de Disco	<code>parse_lsblk()</code> , <code>parse_smartctl()</code> , <code>resolve_disk_type</code>	<code>disk_*</code>

	<code>() ,</code> <code>resolve_disk_inte</code> <code>rface() ,</code> <code>resolve_partition</code> <code>_role()</code>	
Parsers de Rede	<code>parse_ip_addr() ,</code> <code>parse_ip_link() ,</code> <code>parse_default_gat</code> <code>eway() ,</code> <code>build_network_int</code> <code>erface()</code>	<code>network_*</code>
Parsers de SO	<code>parse_os_release(</code> <code>) , parse_uname() ,</code> <code>parse_uptime_seco</code> <code>nds() ,</code> <code>clean_hostname() ,</code> <code>clean_timezone()</code>	<code>system_*</code>
Parsers de Placa-mãe	<code>parse_baseboard(</code> <code>) , parse_bios() ,</code> <code>parse_cpu_sockets</code> <code>() ,</code> <code>parse_memory_slot</code> <code>s_summary() ,</code> <code>parse_system_slot</code> <code>s()</code>	<code>motherboard_*</code>
Conversão de unidades	<code>kb_to_bytes() ,</code> <code>kb_to_gb() ,</code> <code>kb_to_mb() ,</code> <code>bytes_to_gb() ,</code> <code>bytes_to_tb() ,</code> <code>safe_float() ,</code> <code>safe_int()</code>	Geral
Sanitização	<code>_clean() ,</code> <code>sanitize_string(</code> <code>) ,</code>	Geral

	<code>read_sysfs_khz_to_mhz()</code>	
--	--------------------------------------	--

Arquivo: `agent/utils/shell.py`

Função	Descrição
<code>run(cmd)</code>	Executa comando com <code>LC_ALL=C</code> . Retorna <code>stdout</code> ou <code>None</code> em qualquer falha. Timeout: 10s.
<code>run_permissive(cmd)</code>	Retorna <code>stdout</code> mesmo com exit code não-zero. Usado para <code>smartctl</code> , que retorna códigos não-zero com dados válidos. Timeout: 15s.

`LC_ALL=C` é obrigatório — garante saída em inglês independente do locale do sistema. Sem isso, `lscpu` em português retorna `"Arquitetura"` em vez de `"Architecture"` , quebrando os parsers silenciosamente.

Arquivo: `agent/utils/serializer.py`

```
1 def to_json(data):
2     return json.dumps(data, indent=4)
3
```

9. Módulo: sender

Arquivo: `agent/utils/sender.py`

Responsável por enviar os payloads ao backend central via HTTP POST com autenticação Bearer Token.

```
1 URL = "" # Configurar antes de ativar o envio
2 TOKEN = "seu_token_aqui"
3
```

Cabeçalho	Valor
<code>Content-Type</code>	<code>application/json</code>

Authorization

Bearer {TOKEN}

Tratamento de erros: `ConnectionError`, `Timeout` e exceções genéricas são capturadas e impressas sem travar o agente.

Status atual: as chamadas `send_data()` estão comentadas no `__main__.py`. Os payloads são impressos no console via `print(json.dumps(...))`. Para ativar o envio, descomentar as linhas `# send_data(...)` e configurar `URL` e `TOKEN` em `sender.py`.

10. Organização como Módulo Python

O agente é organizado como um pacote Python instalável, permitindo execução direta via `python -m agent` ou geração de executável com PyInstaller.

`pyproject.toml`

```
1 [project]
2 name = "agent-monitor"
3 version = "1.0.0"
4
5 [tool.setuptools.packages.find]
6 where = ["src"]
7
8 [project.scripts]
9 agent = "agent.__main__:main"
10
```

`where = ["src"]` instrui o `setuptools` a buscar pacotes dentro de `src/`. A entrada `[project.scripts]` cria o comando `agent` no PATH após instalação, apontando para `main()` em `src/agent/__main__.py`.

Instalação em modo editável (desenvolvimento)

```
1 cd Agent/
2 python -m venv .ambiente_venv
3 source .ambiente_venv/bin/activate
4 pip install -e .
5
```

Com `pip install -e .`, o Python aponta diretamente para o código em `src/`. Alterações em `.py` têm efeito imediato sem reinstalação. O comando `agent` fica disponível no PATH do `venv`.

Execução direta (sem instalar)

```
1 cd Agent/
```

```
2 python -m agent
3
```

Requer que `src/` esteja no `PYTHONPATH` ou que o pacote esteja instalado.

11. Geração do Executável com PyInstaller

O agente é distribuído como um binário Linux único, sem dependência de Python instalado na máquina alvo.

Entry point do PyInstaller

O arquivo `Agent_Exec/main.py` é minimal — apenas importa e chama `main()` :

```
1 # Agent_Exec/main.py
2 from agent.__main__ import main
3
4 if __name__ == "__main__":
5     main()
6
```

Essa separação existe porque o PyInstaller precisa de um arquivo `.py` como entrada, mas o código real fica em `src/agent/__main__.py`, organizado como módulo Python.

Comando de geração

O comando está documentado em `agent/global_information/cria_executavel.txt` :

```
1 pyinstaller --onefile -n agent \
2   --paths src \
3   --distpath Agent_Exec/dist \
4   --workpath Agent_Exec/build \
5   Agent_Exec/main.py
6
```

Flag	Descrição
<code>--onefile</code>	Empacota tudo em um único binário autocontido
<code>-n agent</code>	Nome do executável gerado
<code>--paths src</code>	Adiciona <code>src/</code> ao <code>PYTHONPATH</code> interno do PyInstaller
<code>--distpath</code>	Diretório de saída do binário final (<code>Agent_Exec/dist/</code>)

<code>--workpath</code>	Diretório de arquivos temporários (<code>Agent_Exec/build/</code>)
-------------------------	---

Resultado e distribuição

O executável final em `Agent_Exec/dist/agent` tem aproximadamente 11-12 MB e inclui o interpretador Python, todas as bibliotecas (`psutil` , `requests` , etc.) e o código do agente.

```

1 | chmod +x Agent_Exec/dist/agent
2 | ./Agent_Exec/dist/agent
3 |
4 | # Com instalação automática de dependências:
5 | ./Agent_Exec/dist/agent --install-deps
6 |

```

Compatibilidade: o binário gerado pelo PyInstaller é vinculado à versão do `glibc` da máquina de compilação. Para máxima compatibilidade com diferentes distribuições Linux, compilar na distribuição mais antiga que se deseja suportar (ex: Ubuntu 20.04 ou Debian 11).

12. Dependências

Bibliotecas Python

Biblioteca	Uso
<code>psutil</code>	Coleta de métricas de CPU, memória, disco e rede
<code>requests</code>	Envio de payloads HTTP ao backend
<code>pyinstaller</code>	Geração do executável Linux autocontido

Ferramentas do sistema (Linux)

Ferramenta	Uso	Requisito
<code>lscpu</code>	Detecção de virtualização e dados de CPU	Obrigatório

<code>lsblk</code>	Lista de discos e partições	Obrigatório
<code>ip</code>	Interfaces de rede e rotas	Obrigatório
<code>dmidecode</code>	Hardware físico (placa-mãe, RAM, BIOS)	Requer root
<code>smartctl</code>	Saúde e metadados de disco físico	Requer root
<code>ethtool</code>	Speed/duplex/driver de interface de rede física	Opcional
<code>lspci</code>	Identificação de chipset	Opcional
<code>timedatectl</code>	Fuso horário	Sistemas com systemd
<code>journalctl</code>	Logs do sistema	Sistemas com systemd

13. Resumo dos Módulos

Módulo	Arquivo principal	Função	Frequência
<code>global_information</code>	<code>global_information.py</code>	Identidade persistida do agente e do host (IDs, hostname, IP, ambiente)	Uma vez por tipo de payload
<code>discovery/system</code>	<code>system_discovery/system.py</code>	Sistema operacional, kernel, fuso horário, uptime	Uma vez
<code>discovery/cpu</code>	<code>cpu_discovery/cpu.py</code>	Modelo, frequências, topologia de CPUs lógicas	Uma vez

discovery/mem	mem_discovery/mem.py	RAM total e slots físicos (físico) ou totais+swap (VM)	Uma vez
discovery/disk	disk_discovery/disk.py	Discos, partições, S.M.A.R.T. (físico) ou lsblk (VM)	Uma vez
discovery/network	network_discovery/network.py	Interfaces, IPs, gateway, hardware (físico) ou IPs (VM)	Uma vez
discovery/motherboard	motherboard_discovery/motherboard.py	Placa-mãe, BIOS, chipset, slots (null em VM)	Uma vez
discovery/tools	tools_discovery/tools.py	Verificação e instalação de ferramentas do sistema	Uma vez (antes dos demais)
coleta/cpu	cpu_coleta/cpu.py	Percentual de uso da CPU	A cada 5 segundos
coleta/mem	mem_coleta/mem.py	Uso de memória RAM	A cada 5 segundos
coleta/disk	disk_coleta/disk.py	Uso do disco raiz	A cada 5 segundos
coleta/network	network_coleta/network.py	Contadores de bytes enviados/recebidos	A cada 5 segundos
coleta/logs	logs_coleta/logs.py	Últimas 10 linhas do auth.log	A cada 5 segundos

Próximos passos documentados no código

`logs_coleta` : suporte a `syslog` , `dmesg` e `journalctl` ; controle de posição de leitura por inode para evitar duplicatas; detecção de rotação de logs; limitação por janela de tempo.

`sender.py` : ativar envio HTTP descomentando as chamadas em `__main__.py` e configurando `URL` e `TOKEN` .